

Phase 2 Deep Dive: Direct Preference Optimization (DPO) & Verifiable Data Pipeline

Author: Monthir Ali

Companion File: [study-plan-post-training.md](#)

Target Topics: Bradley-Terry Preference Formulation, DPO Loss Derivation, Implicit Reward Scaling (β), On-Policy Verifiable Data Generation, and Why DPO Requires SFT Initialization.

PHASE 2: DPO DEEP DIVE	
1. Problem Formulation	→ Post-SFT Alignment: Why SFT alone is insufficient
2. Mathematical Theory	→ Bradley-Terry, Implicit Rewards, Closed-Form DPO Loss
3. Data Pipeline	→ On-Policy Sampling, Compiler Ground Truth (0/1 Reward)
4. Optimization Dynamics	→ Beta (0.1), Reference Model Anchoring, Reward Margins
5. The Critical Finding	→ Why DPO from Base fails (The SFT Warm-Start Dependency)

1. Problem Formulation: Why Preference Optimization After SFT?

Supervised Fine-Tuning (SFT) trains a model via maximum likelihood estimation (MLE) on gold-standard demonstrations (x, y^*) :

$$\max_{\theta} \sum_{(x,y^*) \in \mathcal{D}} \log \pi_{\theta}(y^* \mid x)$$

Limitations of SFT Alone:

- 1. Mode Averaging vs Mode Selection:** SFT treats all tokens in the dataset as equally positive targets. If the dataset contains subtle suboptimal patterns or stylistic noise, SFT blindly imitates them.
- 2. No Negative Signal:** SFT never teaches the model *what not to do*. If the model makes a minor constraint violation (e.g. passing invalid parameter ordering to `sp.AtMostKInARow`), SFT provides no direct gradient pushing probability *away* from that specific failure mode.
- 3. Exposure Bias:** SFT trains on teacher-forced prefixes $y^*_{<t}$. During auto-regressive generation at inference time, the model conditions on its own prior generated tokens. Small distribution shifts can

compound into invalid code.

The Role of Preference Optimization: To expose the model to its own mistakes, reward valid execution (y_w), and actively penalize invalid execution (y_l) on identical prompt inputs x .

2. Mathematical Formulation: From RLHF to DPO

A. The Bradley-Terry Preference Model

Given a prompt x and two candidate completions (y_w, y_l) where $y_w \succ y_l$ (y_w is preferred over y_l), the probability that a human (or compiler) prefers y_w is parameterized by a latent reward function $r(x, y)$:

$$P(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l)) = \frac{1}{1 + e^{-(r(x, y_w) - r(x, y_l))}}$$

B. Standard RLHF (PPO) and Its Bottlenecks

Traditional RLHF (Ouyang et al., 2022) optimizes the policy π_θ against a learned reward model $r_\phi(x, y)$ subject to a KL-divergence penalty preventing deviation from the reference policy π_{ref} :

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta} [r_\phi(x, y)] - \beta \mathbb{D}_{\text{KL}}(\pi_\theta(y \mid x) \parallel \pi_{\text{ref}}(y \mid x))$$

Why PPO is difficult in practice: It requires holding **4 separate models in VRAM** simultaneously (Actor π_θ , Critic V_ψ , Reference π_{ref} , and Reward Model r_ϕ), alongside high-variance reinforcement learning training dynamics.

C. The DPO Reparameterization (Rafailov et al., 2023)

Rafailov et al. proved that the optimal policy π^* under the KL-constrained RL objective has an exact closed-form relationship to the ground-truth reward:

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

Rearranging for the reward function $r(x, y)$:

$$r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x)$$

Substituting this implicit reward directly into the Bradley-Terry preference model causes the partition function $Z(x)$ to cancel out:

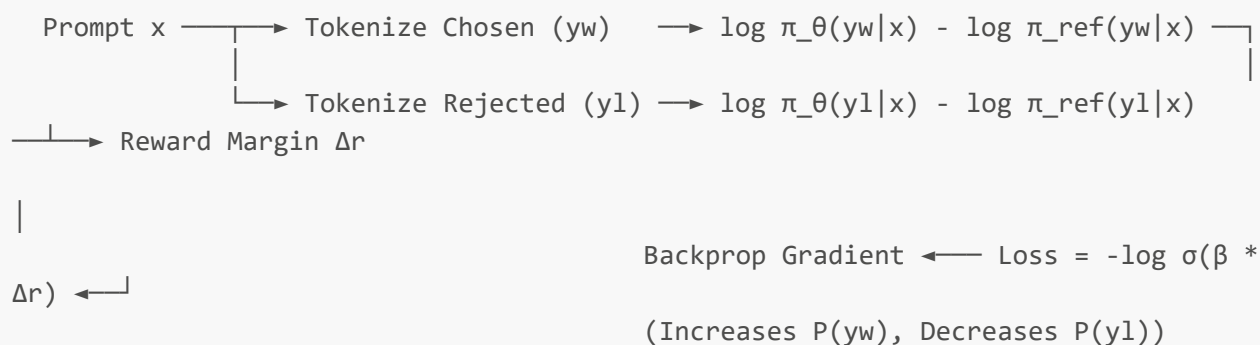
$$r(x, y_w) - r(x, y_l) = \beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}$$

D. The Final DPO Objective

The policy π_θ can now be optimized directly using binary cross-entropy on preference pairs without ever training an explicit reward model:

$$\mathcal{L}_{\text{DPO}}(\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right]$$

THE DPO GRADIENT MECHANISM

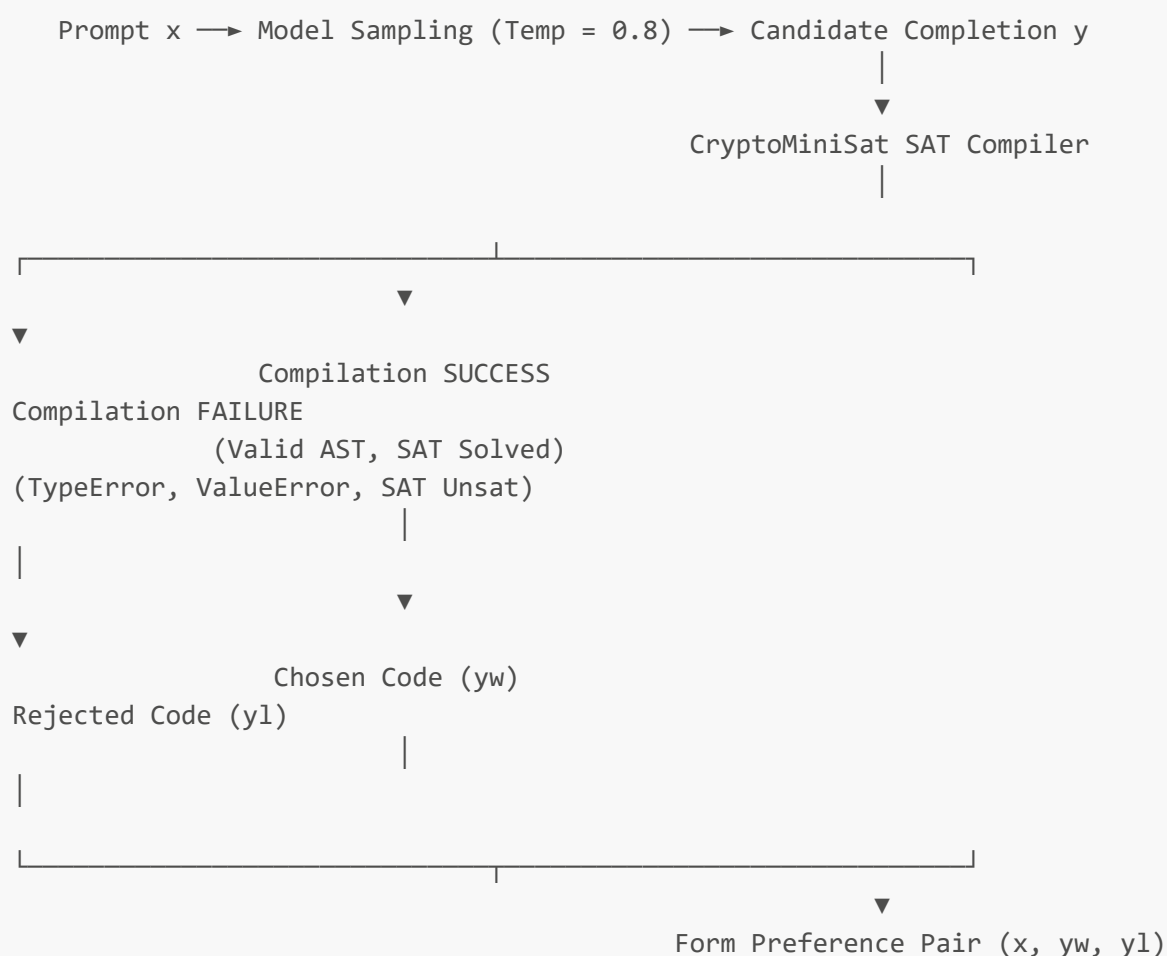


3. Data Pipeline: Compiler-Grounded Verifiable Preference Generation

A major weakness in general NLP preference datasets (like UltraFeedback or HH-RLHF) is **noisy, subjective human labels or biased LLM-as-a-judge evaluations**.

In our pipeline ([src/generate_dpo_pairs.py](#)), preference labels are 100% **verifiable and objective**:

VERIFIABLE DPO DATA PIPELINE



1. We sample candidate completions $y \sim \pi(y \mid x)$ at temperature $T = 0.8$.

- 2. Each sample is executed in an isolated process against the SweetPea SAT solver.
- 3. If y produces a valid trial sequence, it is marked as **Chosen** (y_w). If y raises a syntax error, invalid signature, or UNSAT constraint violation, it is marked as **Rejected** (y_l).
- 4. We generated 300 verified preference pairs (250 train, 50 validation).

4. Hyperparameter Dynamics in DPO

Parameter	Value	Why this value?
Beta (β)	0.1	Controls how strictly the policy is anchored to π_{ref} . If $\beta \rightarrow 0$, the reference model is ignored, leading to policy collapse. If $\beta \rightarrow \infty$, the policy cannot move. $\beta = 0.1$ is the empirical gold standard for code and reasoning tasks.
Learning Rate	5.0e-5	DPO learning rate is $4\times$ smaller than SFT (2.0×10^{-4}). Preference optimization operates on log-ratio gradients; high learning rates cause catastrophic forgetting of syntax.
Loss Type	sigmoid	Standard Bradley-Terry logistic loss $\log \sigma(\beta \cdot \Delta r)$.
Reference Model	Implicit / Frozen Base	By utilizing PEFT LoRA, the base model weights serve as the frozen π_{ref} with zero extra memory overhead, while adapter weights represent π_θ .

Metrics to Track During DPO Training:

- 1. **rewards/margins** ($\beta \log \frac{\pi_\theta(y_w)}{\pi_{\text{ref}}(y_w)} - \beta \log \frac{\pi_\theta(y_l)}{\pi_{\text{ref}}(y_l)}$): Must steadily increase. In our run, margin grew from $+0.045 \rightarrow +14.10$.
- 2. **rewards/accuracies** (**fraction where $r(y_w) > r(y_l)$**): Jumped from $0.475 \rightarrow 1.000$ (100% preference classification).
- 3. **logs/chosen vs logs/rejected**: Chosen log-probabilities increase (e.g. $-168 \rightarrow -104$) while rejected log-probabilities decrease (e.g. $-204 \rightarrow -281$).

5. The Critical Finding: Why DPO Fails Without SFT Warm-Start

When we executed DPO directly on the Base Model ([checkpoints/dpo_qwen_1.5b](#)), the optimization metrics appeared perfect:

- Train loss dropped from $0.67 \rightarrow 0.0702$.
- Reward margin reached $+14.10$.
- Validation preference accuracy reached **100.0%**.

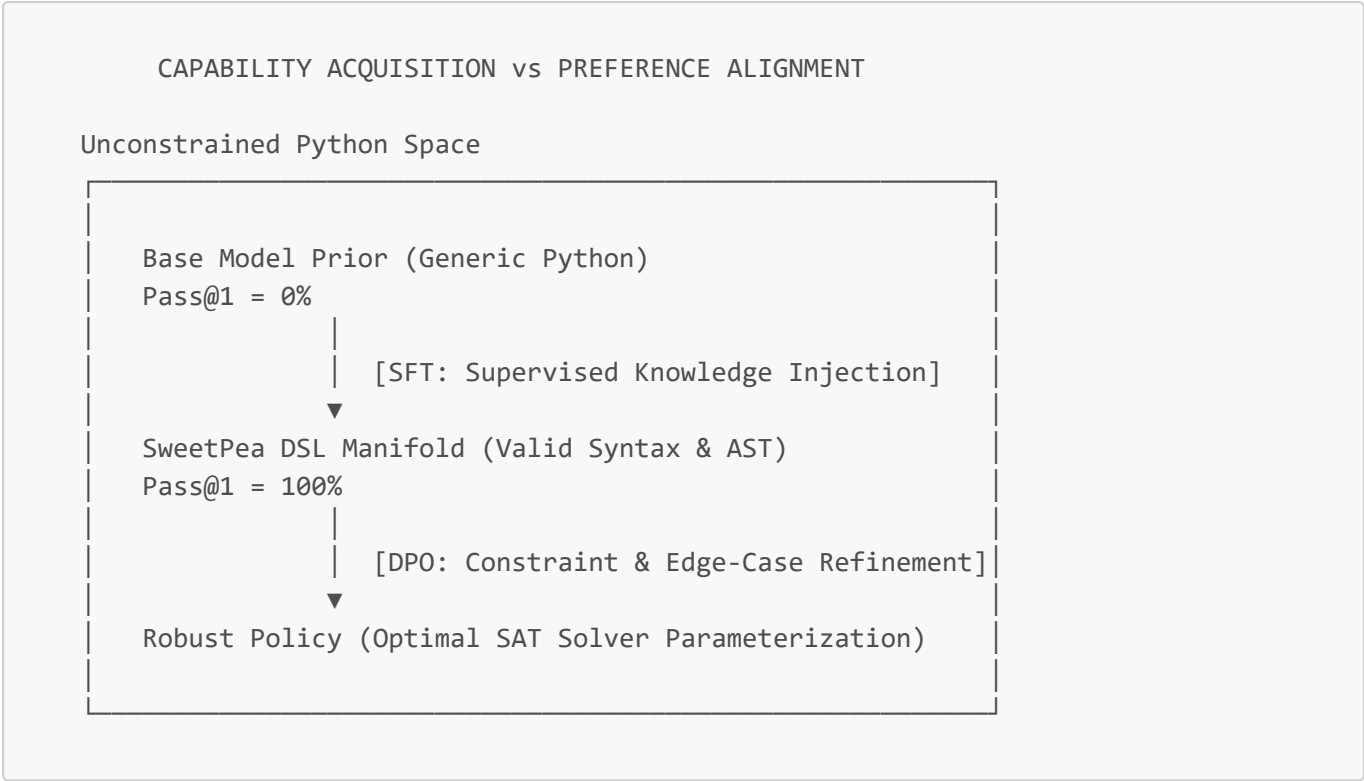
Yet, on the held-out test set, compiler Pass@1 remained **0.00% (0/150)**.

Why did this happen?

- 1. **DPO is a Ranking Operator, Not a Grammar Teacher**: DPO shifts probability mass between two candidate sequences *relative to each other*. If the base model starts with zero probability on valid SweetPea syntax, penalizing invalid sequences simply redirects probability mass to *other* invalid sequences (in our run, shifting errors from **ValueError** to **AttributeError**).

2. The Post-Training Sequence is Strictly Hierarchical:

- **Phase 1 (SFT):** Learns the valid syntactic manifold and domain grammar (0% → 100% Pass@1).
- **Phase 2 (DPO / RL):** Learns constraint nuance, edge-case robustness, and style preferences within the valid manifold.



6. Codebase Architecture Map

Component	Path	Functionality
Preference Sampler	src/generate_dpo_pairs.py	High-throughput batched sampling at $T = 0.8$ with in-memory CryptoMiniSat verification.
DPO Dataset	data/dpo_train.jsonl, data/dpo_val.jsonl	300 verifiable (x, y_w, y_l) records with error annotations.
DPO Training Pipeline	src/train_dpo.py	PEFT-backed LoRA optimization using HuggingFace TRL <code>DPOTrainer</code> .
DPO Config	configs/dpo_qwen_1.5b.yaml	Reproducible configuration ($\beta = 0.1$, LR 5×10^{-5} , effective batch size 8).